

SOFTWARE REQUIREMENTS SPECIFICATION

Blood Cell Classification Using Convolutional Neural Network (CNN)

Project Type

Deep Learning & Computer Vision

Dataset

Blood Cell Detection Dataset

Prepared By

Rimsha Pervaiz

Technologies

Python • TensorFlow • Keras • Jupyter Notebook

Document Overview

This Software Requirements Specification (SRS) defines the functional and non-functional requirements, system architecture, processing workflow, evaluation criteria, and limitations of the Blood Cell Classification Using CNN project.

Table of Contents

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. CNN Model Requirements
7. System Evaluation and Results
8. Constraints and Limitations
9. Future Enhancements
10. Conclusion
11. References

1. Introduction

1.1 Purpose

The purpose of this project is to develop an automated blood cell classification system using a Convolutional Neural Network (CNN). The system processes annotated microscopic blood-smear images, extracts individual cells, and classifies them into Red Blood Cells (RBC) and White Blood Cells (WBC).

1.2 Scope

The project covers dataset loading, annotation processing, cell extraction, image preprocessing, CNN training, validation, testing, performance visualization, and trained-model storage. The current implementation performs classification on individual cropped cells rather than complete full-image object detection.

1.3 Objectives

- Process the Blood Cell Detection Dataset and its annotations.
- Extract individual blood cell images from bounding boxes.
- Classify cells into RBC and WBC categories using a CNN.
- Evaluate performance using accuracy, loss, precision, recall, F1-score, and a confusion matrix.
- Save the trained model for future inference and deployment.

1.4 Definitions and Abbreviations

Term	Meaning
CNN	Convolutional Neural Network
RBC	Red Blood Cell
WBC	White Blood Cell
SRS	Software Requirements Specification
Keras	High-level deep learning API used with TensorFlow

2. Overall Description

2.1 Product Perspective

The project is implemented as a notebook-based deep learning system. Images and annotations are processed to create labeled cell crops. Each image is resized to 64 × 64 RGB pixels, normalized, and passed to a CNN classifier. The trained network outputs class probabilities and the final model is stored in .keras format.

2.2 Product Functions

- Load images and annotations.
- Read image names, coordinates, and class labels.
- Crop individual cells using bounding boxes.
- Resize and normalize images.
- Split data into training, validation, and testing sets.
- Apply class weights to reduce imbalance effects.

- Train and evaluate a CNN.
- Generate performance graphs, classification reports, and confusion matrices.
- Display sample predictions with confidence scores.
- Save the trained model as `blood_cell_cnn.keras`.

2.3 Operating Environment

Component	Requirement
Programming Language	Python 3
Execution Environment	Jupyter Notebook / Kaggle Notebook / Google Colab
Deep Learning	TensorFlow and Keras
Data Processing	NumPy and Pandas
Visualization	Matplotlib and Seaborn
Evaluation	Scikit-learn
Image Processing	Pillow (PIL)

3. Functional Requirements

ID	Requirement	Description
FR-01	Dataset Loading	Load blood-smear images and annotation data.
FR-02	Annotation Processing	Read image names, bounding-box coordinates, and class labels.
FR-03	Cell Extraction	Crop individual cells from annotated bounding boxes.
FR-04	Image Preprocessing	Resize cells to 64×64 and normalize pixel values to 0–1.
FR-05	Data Splitting	Create stratified training, validation, and testing sets.
FR-06	CNN Training	Train the CNN using convolution, pooling, dropout, dense, and softmax layers.
FR-07	Imbalance Handling	Calculate and apply class weights during training.
FR-08	Model Evaluation	Calculate loss, accuracy, precision, recall, and F1-score.
FR-09	Confusion Matrix	Compare actual labels with predicted labels.
FR-10	Prediction Visualization	Show sample images with true label, predicted label, and confidence.
FR-11	Model Storage	Save the trained CNN in .keras format.

4. Non-Functional Requirements

4.1 Performance

The system should train and evaluate efficiently within the computational resources available in the selected notebook environment and provide strong predictive performance on unseen test samples.

4.2 Reliability

Fixed random seeds, stratified splitting, early stopping, and best-weight restoration are used to improve reproducibility and reduce overfitting.

4.3 Usability

The notebook is organized into clearly labeled sections so that the complete workflow can be executed step by step.

4.4 Maintainability and Portability

Key parameters such as image size, epochs, batch size, and model layers can be modified easily. The project can run in compatible Python notebook environments when the dataset and required libraries are available.

5. System Architecture

Input → Blood-smear images and annotations

Preprocessing → Annotation parsing → Cell cropping → Resizing → Normalization

Data Preparation → Stratified train/validation/test split → Class weighting

Model → Convolution → Pooling → Convolution → Pooling → Convolution → Pooling → Dropout → Dense → Softmax

Output → RBC/WBC prediction, confidence score, evaluation metrics, confusion matrix, and saved model.

6. CNN Model Requirements

The CNN accepts $64 \times 64 \times 3$ RGB images. The architecture contains convolutional layers with 32, 64, and 128 filters, max-pooling layers for feature reduction, dropout layers to reduce overfitting, a dense layer with 128 neurons, and a softmax output layer. Training uses the Adam optimizer and sparse categorical cross-entropy loss.

7. System Evaluation and Results

The completed model was evaluated on **351 unseen test cell images**. It achieved a test accuracy of **99.72%** and a test loss of **0.0188**.

Metric	Result
Test Loss	0.0188
Test Accuracy	99.72%
Correct Predictions	350 / 351
RBC Precision	99.70%
RBC Recall	100.00%

Metric	Result
RBC F1-Score	99.85%
WBC Precision	100.00%
WBC Recall	93.00%
WBC F1-Score	96.55%

7.1 Confusion Matrix Interpretation

All 336 RBC test samples were correctly classified as RBC. Out of 15 WBC samples, 14 were correctly classified as WBC and 1 was incorrectly predicted as RBC. Therefore, the system produced 350 correct predictions out of 351 test samples.

8. Constraints and Limitations

- The dataset is imbalanced, with more RBC samples than WBC samples.
- The current implementation classifies cropped cells rather than detecting cells directly in a complete blood-smear image.
- Performance may change for images obtained under different staining, microscope, or imaging conditions.
- The available dataset size and diversity may limit generalization to real clinical environments.

9. Future Enhancements

- Apply stronger data augmentation and collect more WBC samples.
- Use transfer learning with pre-trained image models.
- Extend the classifier to additional blood cell categories.
- Implement end-to-end object detection on complete blood-smear images.
- Deploy the model through a Streamlit web application.

10. Conclusion

The Blood Cell Classification Using CNN project provides an automated deep learning workflow for distinguishing between Red Blood Cells and White Blood Cells. The system successfully processes annotated images, extracts individual cells, performs preprocessing, trains a CNN, and evaluates its predictions using standard classification metrics. The achieved **99.72% test accuracy** demonstrates strong performance on the available test data and provides a foundation for future medical image analysis and deployment work.

11. References

1. Blood Cell Detection Dataset, Kaggle.
2. TensorFlow Documentation.
3. Keras Documentation.
4. Scikit-learn Documentation.